Blick - Project Documentation

Product concept, functional specification, verified data sources, architecture, privacy, and delivery scope

Document status: Android-first MVP specification — see "Current implementation status" immediately below for what already exists in this repository versus what the rest of this document specifies as still planned.\ **Updated:** 30 July 2026

Current implementation status (as of 30 July 2026)

This document specifies the full intended product. Most of the sections below describe that end-state design in the present tense, as a specification does — they are **not** a claim that all of it already exists. This section is the authoritative summary of what is actually built today; where the two disagree, this section wins.

Implemented today (Android client + backend):

- Live SL stop search and transport-mode/line/direction discovery during routine setup.
- The routine-creation wizard, backed by Room persistence.
- A foreground routine details/live-preview screen: manual "Refresh" only, showing **up to two departures total** (not three — see the corrected departure-count language later in this document).
- Routine editing (the same wizard, in an edit mode), enable/disable, pause-today / resume-today (with automatic cleanup of an expired pause), and deletion behind an in-screen Material 3 confirmation dialog.
- A first-beta one-routine limit enforced at the application/UI level.
- Loading, live, no-departures, offline, stale, and unavailable states for the departures section — including a fix ensuring that editing a routine to a different site/line/direction/transport-mode, followed by that new configuration's first fetch failing, can never surface the *previous* configuration's departures mislabelled as "stale" data for the new one. The client's retained last-successful snapshot is now scoped to the exact departure identity (site, line, direction, transport mode) that produced it, and is discarded rather than reused across an identity change.
- The backend's full contract, request validation, upstream normalization, and caching logic (183 passing automated tests as of this update).

Not yet implemented (described in the sections below purely as the plan):

- Automatic activation of a routine at its configured start time.
- Any periodic/background refresh, including the planned 30-second active-routine loop.
- The ongoing, auto-updating lock-screen notification.
- Runtime notification-permission onboarding. The `POST_NOTIFICATIONS` manifest entry is declared in preparation for this, but the application does not request it at runtime anywhere yet — there is no notification feature to gate it behind.
- Automatic removal of the notification at the end of a routine's active window.
- Scheduling reactions to routine changes or a device reboot.
- Persistent stale-data storage beyond the current screen's in-memory session.
- The home-screen widget.

1. Project summary

Blick is an Android-first application that automatically displays upcoming SL bus, metro, commuter-train, tram, local-rail, and compatible ferry departures during times chosen by the user.

Instead of requiring the user to open a journey-planning application and search for the same stop every morning, Blick places the relevant information in one quiet, updating Android notification. The user selects an SL site, transport mode, line, direction, active weekdays, and a time window. During that window, the application shows the next relevant departures and disruptions affecting the selection.

The first release is a native Android application. A native iPhone client is planned for a later phase and will use the same platform-neutral backend contract. A Smart TV departure display is a possible future feature, not part of the MVP.

The application is intended to be a calm information display. It does not tell users when to leave, predict whether they will catch a departure, or use stressful messages such as "Leave now" or "Probably too late."

2. Problem

Many commuters repeatedly check the same information:

- the same bus stop or station;
- the same line and direction;
- at approximately the same time;
- on the same weekdays.

Existing journey-planning applications usually require several actions: unlock the phone, open an application, search for a stop, station, or journey, and inspect the results. This is unnecessary when the regular commute is already known.

Blick reduces this repeated interaction by showing the information automatically when it is relevant.

3. Proposed solution

The user creates a scheduled commute routine, for example:

Setting	Example
Site	Fruängen
Transport	Metro
Line	14

Direction	Toward T-Centralen
Days	Monday-Friday
Active period	07:30-08:00

At 07:30, a quiet, ongoing notification appears:

> **Fruängen to T-Centralen**
> Next: 4 min
> Then: 11 min / 18 min

The same notification updates during the selected period. When the first departure is no longer relevant, the following departure becomes the next one. At 08:00, the notification disappears and active updating stops.

When there is a relevant disruption, it is added below the departures:

> **Fruängen to T-Centralen**
> Next: 4 min
> Then: 11 min / 18 min
> Disruption: Delays on line 14 due to a signal fault

If no relevant disruption exists, the disruption area is not shown.

---

4. Product principles

Calm

The application presents neutral information without pressure, judgement, repeated alerts, or urgency-based language.

Automatic

Once a routine has been configured, the user should not need to open the application every day.

Relevant

Only departures and disruptions connected to the selected site, transport mode, line, and direction should be displayed.

Glanceable

The most important information must be understandable within a few seconds from the lock screen or notification panel.

Respectful of attention

The application updates one ongoing notification instead of sending a new notification every few minutes.

Honest about data

The application distinguishes scheduled information, real-time information, stale information, and unavailable information. It does not invent missing fields or present old data as current.

---

5. Supported SL transport

The first version is intended to support:

- buses;
- metro;
- commuter trains;
- local and light rail;
- trams;
- SL ferry departures when compatible data is available.

The verified upstream transport-mode values are `BUS`, `METRO`, `TRAM`, `TRAIN`, `SHIP`, `FERRY`, and `TAXI`. The Android UI will expose only modes included in the product scope.

The same routine model applies to each supported mode: the user selects an SL site, a line where relevant, a direction, active days, and an active time window.

---

6. Target users

The initial target users are regular SL commuters who:

- travel from the same site at a similar time on workdays or study days;
- want to check departures while preparing at home;
- prefer passive information over repeatedly searching in a journey planner;
- want relevant disruption information without unrelated network-wide announcements.

The first product is SL-specific. Support for other public-transport operators is a possible later expansion and is not part of the current architecture contract.

---

7. Core user experience

Initial setup

Steps 1–7 are implemented today. Step 8 (notification permission) is still planned — see "Current implementation status" above.

1. The user opens the Android application.
2. The user searches for and selects an SL site.
3. The user selects a supported transport mode.
4. The user selects a line and direction.

5. The user selects active weekdays.
6. The user selects a start and end time.
7. The user saves the commute routine.
8. The application requests notification permission and explains lock-screen visibility.

Line and direction options are initially discovered from live departures at the selected site. The SL Transport departures endpoint exposes only services inside its current forecast window. A route that is not currently operating may therefore be unavailable during setup. This is a documented MVP limitation. Direction discovery must remain behind an application interface so a more complete source can replace it later without changing the saved-routine model.

Daily operation

This entire section describes the planned automatic/scheduled and notification behavior. None of it is implemented yet — the current behavior is a manually-opened, foreground-only details screen with a manual Refresh action (see "Current implementation status" above).

1. The routine becomes active around the configured start time.
2. The application requests current normalized departures from the Blick backend.
3. It filters by the saved site, line, transport mode, and direction code.
4. One ongoing notification appears.
5. The notification is refreshed during the active period.
6. Relevant disruption information is included when available.
7. The notification is removed at the configured end time.

User controls

The notification or application may offer:

- **Refresh**
- **Pause today**
- **End now**
- **Open routine**

These controls should remain optional and visually secondary.

---

8. Functional requirements

Commute routines

The application must allow the user to:

- create a commute routine;
- choose an SL site;
- choose a supported transport mode;

- choose a line and direction;
- choose one or more weekdays;
- set a start and end time;
- enable or disable a routine;
- edit or delete a routine;
- pause a routine for the current day.

The first UI supports one routine. The local database is designed for multiple routines so later expansion does not require replacing the storage model.

Departure display

During an active routine, the application must:

- retrieve current departures through the versioned backend API;
- display up to two upcoming departures total, when available;
- show the selected site, line, and direction;
- calculate the visible countdown locally;
- automatically remove departures whose effective time has passed;
- update the existing notification rather than creating repeated notifications;
- show when the source response was fetched when data may be stale;
- distinguish real-time and scheduled-only information;
- show cancellations when they can be determined from verified upstream fields;
- handle missing, malformed, or unavailable data safely.

Disruptions

The application must:

- retrieve detailed SL disruption messages through the backend;
- match disruptions using the same SL site and line namespace as departures;
- select a Swedish message variant when available;
- display relevant disruptions in neutral language;
- avoid unrelated network-wide messages;
- show cancellations, changed routes, station closures, and significant delays when provided;
- hide the disruption section when nothing relevant is reported.

Notification and lock screen

Planned — not implemented yet. Runtime notification-permission onboarding is pending; see "Current implementation status" above.

The application must:

- request Android notification permission;
- use one ongoing notification during the active period;

- permit full departure information on the lock screen when allowed by the user's system settings;
- remove the notification when the routine ends;
- explain that Android and the user's privacy settings control lock-screen visibility.

Future widget

A home-screen widget is outside the first MVP. A later Android version may add a widget that displays the saved route and next departures, provides manual refresh, and opens the routine when tapped.

---

9. Verified data sources

SL Transport API

SL Transport is the primary source for:

- the complete list of SL sites;
- site coordinates and child stop-area IDs;
- stable SL line IDs;
- directions and direction codes;
- scheduled and expected departure times;
- stop-area and stop-point information;
- journey and departure states;
- embedded trip and site deviation signals.

The backend uses:

- `GET /v1/sites?expand=true`
- `GET /v1/sites/{siteId}/departures`

The API is currently keyless. Fair-use requirements still apply.

Official documentation: SL Transport

SL Deviations API

SL Deviations provides detailed disruption records, including:

- stable deviation-case IDs;
- created and modified timestamps;
- publication windows;
- priority fields used for sorting;
- Swedish and other message variants;
- affected stop areas and lines.

The backend uses:

- `GET /v1/messages`

SL Transport Site IDs have been verified as accepted by the SL Deviations `site` filter. Line IDs and stop-area IDs also align with their corresponding entities in SL Deviations responses. No Trafiklab Stop Lookup conversion is required. The Deviations service is keyless but its documentation asks consumers to request data no more than once per minute.

Official documentation: SL Deviations

Excluded upstream APIs

Trafiklab Timetables and Trafiklab Stop Lookup are not used by the SL-only MVP. Removing them avoids incompatible ID namespaces and the absence of a stable numeric line ID in the Timetables route model.

Licence and attribution

Before public release, the operator must satisfy the current SL API terms, including any required registration and acceptance. The product must visibly state that its transport information is based on data retrieved from Trafiklab.se. When practicable, the attribution should link to Trafiklab.se.

Recommended attribution:

> Based on information from Trafiklab.se

The application must not imply endorsement by or affiliation with SL, Trafiklab, or Samtrafiken. The first version uses its own visual identity and plain transport labels rather than protected logos or copied brand styling.

Official terms: SL API licence

---

10. Technical architecture

Repository

The project uses one repository with separate areas:

```
blick/
 android/
 backend/
 docs/
```

The Android application and backend have separate build systems and tests. `docs/api-contract.md` holds the detailed machine-facing contract. This project documentation remains the product and architecture overview.

Native Android application

The Android client uses:

- Kotlin;
- Jetpack Compose and Material 3;

- MVVM with repository interfaces;
- Hilt for dependency injection;
- Room for commute routines;
- Preferences DataStore only for small application settings;
- Retrofit with kotlinx.serialization for the backend API;
- AlarmManager and WorkManager behind scheduling interfaces;
- Android notification APIs behind a notifier interface.

Pinned SDK values:

```
compileSdk = 36
targetSdk = 36
minSdk = 26
```

`minSdk = 26` is a product support decision. It is not described as a technical requirement for notification channels.

Build toolchain: Android Gradle Plugin 9.2.1 using AGP's built-in Kotlin (Kotlin 2.3.10, no separate `org.jetbrains.kotlin.android` plugin), Gradle 9.4.1, KSP 2.3.9. The Gradle wrapper (`gradlew`, `gradlew.bat`, `gradle-wrapper.jar`, and `gradle-wrapper.properties` with its distribution SHA-256 checksum) is committed, so a fresh clone can build without a pre-existing local Gradle install. `assembleDebug`, `lintDebug`, and `testDebugUnitTest` have not been executed as part of preparing this repository — that environment has no JDK 17, no Android SDK, and no network path to the hosts that would provide either, so these commands are structurally prepared but not build-verified from that environment. See `android/README.md` for the exact toolchain versions and rationale.

Backend

The backend uses:

- TypeScript;
- Hono;
- Zod validation;
- a Vercel-compatible handler;
- a portable application core shared with a local Node development server;
- explicit upstream-client, normalization, cache, and error-handling interfaces;
- Vitest for backend tests.

The backend remains necessary even though the chosen upstream APIs are keyless. It provides:

- a stable, versioned contract for Android and a future iPhone client;
- upstream schema isolation;
- validation and normalization;
- caching and request deduplication;
- fair-use protection;
- consistent timestamps and errors;
- a controlled place to adapt when upstream services change.

No Trafiklab API key is required by the selected endpoints. `.env.example` should therefore contain only genuinely required runtime configuration, such as local port or environment mode.

Platform-neutral data flow

Stage	Responsibility
Android now / iPhone later	Stores routines, schedules active periods, calculates countdowns, and presents information
Blick backend	Validates requests, fetches and normalizes upstream data, applies caching, and returns versioned JSON
SL Transport	Supplies sites, departures, line IDs, direction codes, stop data, and embedded deviation signals
SL Deviations	Supplies detailed and time-bounded disruption messages

The backend must not return Android-specific notification, Room, or UI concepts.

---

11. Backend API contract

All routes are versioned under `/api/v1`.

Envelopes

Successful response:

```
{
 "schemaVersion": 1,
 "data": {}
}
```

Error response:

```
{
 "schemaVersion": 1,
 "error": {
   "code": "VALIDATION_ERROR",
   "message": "A safe, actionable message"
 }
}
```

Supported machine-readable error codes:

```
VALIDATION_ERROR
UPSTREAM_ERROR
```

```
UPSTREAM_TIMEOUT
UPSTREAM_RATE_LIMITED
NOT_FOUND
RATE_LIMITED
INTERNAL_ERROR
```

Errors must not expose stack traces, environment values, or unnecessary upstream details.

Health

`GET /api/v1/health`

Returns service status and a backend timestamp.

Cache policy:

```
Cache-Control: no-store
```

Site search

`GET /api/v1/stops/search?query=`

The backend searches a cached snapshot from SL Transport `/v1/sites?expand=true`.

Normalized site:

```
siteId
name
note
lat
lon
stopAreaIds
```

Search requirements:

- trim and validate the query;
- use case-insensitive and diacritic-tolerant matching;
- search site name and nullable note;
- rank exact, prefix, token-prefix, and substring matches in that order;
- use deterministic tie-breaking;
- return no more than 20 results.

Cache policy:

```
Cache-Control: public, s-maxage=3600, stale-while-revalidate=86400
```

Departures

`GET /api/v1/departures?siteId=`

`siteId` is required and must be a valid positive SL Site ID.

Top-level data:

```
fetchedAt
timeZone
siteId
departures
siteDeviations
```

`timeZone` is always `Europe/Stockholm`. `fetchedAt` records when the backend actually fetched the upstream response, not when a cached response was later served.

Normalized departure:

```
departureId
journey
line
direction
directionCode
destination
via
stopArea
stopPoint
scheduledTime
expectedTime
state
isCancelled
tripDeviations
```

The defensive departure identity is:

```
journey.id + ":" + stopPoint.id + ":" + scheduledTime
```

`expectedTime` is nullable. Clients use `expectedTime` when available and otherwise use `scheduledTime`. The backend never returns `minutesRemaining`; each client calculates it from the effective timestamp and the current device time.

`isCancelled` is true when:

- the departure state is `CANCELLED`; or
- a trip deviation has consequence `CANCELLED`.

Other unfamiliar state strings remain available for forward compatibility and are not reinterpreted as cancellation.

Cache policy:

```
Cache-Control: public, s-maxage=30, stale-while-revalidate=30
```

Disruptions

`GET /api/v1/disruptions?siteId=&lineId=&transportMode=&future=`

`siteId` is required. The remaining filters are optional and validated. `future` defaults to `false`, which limits the normal active-routine request to currently published disruptions. A caller must opt in explicitly when future disruption messages are needed.

Normalized disruption:

```
disruptionId
version
createdAt
modifiedAt
validFrom
validUntil
priority
message
affectedStopAreas
affectedLines
affectedModes
```

`priority` contains the upstream importance, influence, and urgency levels. These values are sorting hints and are not renamed to `severity`.

`message` contains:

```
header
details
scopeAlias
webLink
language
```

The backend selects the message variant whose language is `sv`. It falls back to the first available variant only when no Swedish variant exists.

`affectedModes` is a documented derived list of unique transport modes from affected lines.

Cache policy:

```
Cache-Control: public, s-maxage=60, stale-while-revalidate=60
```

The public contract does not include an unused lines endpoint. One may be added later only when it has a defined consumer, normalized schema, and tests.

---

12. Local information model

Commute routine

A Room `RoutineEntity` supports multiple records even though the first UI exposes one:

```
id
name
siteId
siteName
```

```
transportMode
lineId
lineDesignation
directionCode
destinationLabel
activeDaysMask
startTimeMinutes
endTimeMinutes
enabled
pausedDateEpochDay
```

`siteId`, `lineId`, `transportMode`, and `directionCode` are the stable matching fields. `destinationLabel` is a presentation value and must not be the sole identity — there is no separate `directionLabel` field; direction is represented only by `directionCode`. `activeDaysMask` stores the active weekdays as a bitmask (bit 0 = Monday ... bit 6 = Sunday) rather than a list, and `startTimeMinutes`/`endTimeMinutes` store the active window as minutes-since-midnight. `pausedDateEpochDay` is nullable and holds an epoch-day value only when the routine is paused for a specific date. The entity does not track `createdAt`/`updatedAt` timestamps — routines are identified and ordered by their own fields (currently `name`), not by creation history.

The Room DAO exposes:

```
Flow<List<RoutineEntity>>
getById
upsert
update
delete
deleteById
```

`upsert` is `@Insert(onConflict = OnConflictStrategy.REPLACE)` — an insert that replaces an existing row sharing the same primary key, rather than a separate merge/patch operation.

Preferences DataStore holds only small application settings such as first-launch state, theme choice, and whether an explanation has been dismissed.

Client domain timestamps

Network DTOs receive ISO 8601 strings with explicit offsets. Android maps them into `java.time.Instant` or `ZonedDateTime` domain values instead of retaining raw strings. A future iPhone client will perform an equivalent native conversion.

---

13. Scheduling and Android limitations

Planned — not implemented yet. There is no `WorkManager`/`AlarmManager`/foreground service in the repository today; see "Current implementation status" above.

Android limits background activity to protect battery life. Blick should not attempt to run continuously.

The scheduled commute-window design supports this requirement:

- work begins only around the selected start time;

- data is checked only during the active period;
- one notification is updated instead of producing repeated alerts;
- work stops when the configured period ends.

Perfectly exact background execution may require Android's special exact-alarm permission. Where exact timing is unnecessary, an inexact scheduled start may provide a better balance between reliability, battery use, and permission burden.

Foreground-service use is restricted on modern Android versions and must match current Android and Google Play policy. Scheduling and notification components must be isolated behind interfaces and tested on recent Android versions, including Samsung devices whose battery settings may affect background work.

The application must restore appropriate schedules after device reboot or relevant time-setting changes.

Relevant Android documentation:

- Schedule alarms
- Background work
- Create notifications

---

14. Time, caching, and resilience

Stockholm timestamps

SL Transport emits local Stockholm timestamps without an explicit UTC offset. The backend interprets them using the IANA zone `Europe/Stockholm` and emits ISO 8601 timestamps with an explicit offset.

The conversion must not silently rely on a date library's default daylight-saving-time choice. Tests cover:

- normal winter and summer timestamps;
- both sides of the spring 2026 transition;
- both sides of the autumn 2026 transition;
- the duplicated local hour during the autumn overlap;
- a nonexistent local time during the spring gap.

For an ambiguous autumn timestamp, the backend uses the upstream fetch time and forecast window to select the chronologically plausible occurrence. A malformed or impossible timestamp is handled explicitly rather than silently shifted.

SL Deviations already emits timestamps with offsets; they are validated and preserved.

Serverless caching

The backend uses a `Cache` interface with an in-memory implementation for local development and best-effort per-instance reuse on Vercel.

Important limitations:

- Vercel serverless instances do not share dependable process memory;
- a daily site snapshot in memory is not a guaranteed global daily cache;

- HTTP cache headers are the dependable initial shared layer for public GET responses;
- simultaneous identical requests within one instance should be deduplicated;
- **the SL Deviations one-request-per-minute limit is not currently guaranteed in production, and this is a blocker for public deployment, not a solved problem.** Two compounding gaps: the cache/dedup is per-serverless-instance only (Vercel does not guarantee shared memory across instances), and it is keyed per query combination (site/line/transport-mode/future), so real multi-user traffic spanning many different combinations can exceed one request per minute in aggregate even with the cache working exactly as designed. A shared cache (e.g. Redis) plus a real global rate limiter in front of the SL Deviations call path is required before any significant public traffic — preview/manual-testing-scale usage stays within SL's guidance in practice, but that is not the same as the backend enforcing it.

Stale and unavailable information

When live information cannot be refreshed, the application must not present old data as current. Appropriate states include:

```
Unable to update - last checked 07:42
Scheduled time only
No upcoming departures
```

The backend applies timeouts and consistent error mapping. The Android client retains the last successful response only with a visible stale state and appropriate expiry.

The retained response is scoped to the exact departure identity (site, line, direction, and transport mode) that produced it. If the user edits a routine to a different site/line/direction/mode and that new configuration's first fetch fails, the client must not fall back to the previous configuration's retained response — doing so would mislabel unrelated departures as "stale" data for the new routine. A failed refresh may only fall back to stale data that was captured for the identical configuration currently being fetched.

---

15. Edge cases

The application must account for:

- no upcoming departures;
- expected time missing while scheduled time is available;
- temporarily unavailable real-time information;
- a cancelled departure;
- an unfamiliar future state value;
- a changed destination;
- a changed stop point or platform;
- several stop positions at one site;
- a shortened route;
- a line-wide or mode-wide disruption;
- a disruption without a Swedish message variant;
- the device being offline;

- the phone being restarted;
- notification permission being denied;
- lock-screen content being hidden;
- Android delaying scheduled work;
- a routine spanning midnight;
- daylight-saving-time changes;
- weekday and holiday timetable differences;
- upstream timeouts, malformed data, rate limiting, or service failure;
- a route not appearing during setup because it is outside the current departure forecast.

---

16. Minimum viable product

The MVP contains:

1. Native Android setup application.
2. One routine exposed in the UI.
3. Room persistence designed for multiple routines.
4. SL site search.
5. Bus, metro, commuter-train, and local-rail support.
6. Transport, line, and direction selection from available data.
7. Weekday selection.
8. Configurable start and end time.
9. Up to two departures total when available.
10. One scheduled ongoing notification.
11. Lock-screen visibility subject to Android settings.
12. Relevant SL disruption messages.
13. Cancellation presentation.
14. Automatic removal at the end of the routine.
15. Pause-for-today control.
16. Stale and offline states.
17. Visible Trafiklab attribution.
18. Versioned Hono backend deployed to Vercel.

Outside the MVP:

- user accounts;
- cloud routine synchronization;
- GPS tracking;
- analytics;
- advertising;
- social features;

- arbitrary journey planning;
- iPhone implementation;
- TV implementation;
- casting and device pairing;
- all-Sweden operator coverage;
- walking-time advice;
- predictive "Can I catch it?" messages;
- aggressive alert sounds;
- smartwatch support;
- home-screen widget.

---

17. Planned and possible later versions

Planned: native iPhone client

A native iPhone application is planned after the Android MVP. It will consume the same versioned backend contract and reuse the documented transport identity, filtering, timestamp, and cancellation rules.

The iPhone phase must separately investigate:

- Swift and SwiftUI implementation;
- ActivityKit and Live Activities;
- Apple Push Notification service;
- iOS background-execution constraints;
- local persistence;
- App Store delivery and testing.

The Android notification design must not be assumed to translate directly to iOS.

Possible: Smart TV departure display

A Smart TV display is a possible future feature. The preferred first investigation is a secure web or casting experience using the existing backend, rather than separate applications for every TV operating system.

Potential requirements include:

- a large, remote-friendly departure board;
- secure pairing or a temporary display link;
- explicit consent before sharing a saved routine;
- cross-device configuration without exposing travel routines.

No TV, casting, pairing, or cloud-synchronization code is included now.

Other possible additions

- multiple visible routines;
- separate morning and evening schedules;
- Android home-screen widget;
- holiday and temporary schedule controls;
- improved direction discovery;
- accessibility and theme options;
- Wear OS investigation;
- additional Swedish operators after a separate data-contract review.

---

18. Privacy and security

The first Android application does not require the user's identity or precise location.

MVP privacy choices:

- no account;
- routines stored locally in Room;
- no GPS permission;
- no analytics;
- no advertising identifiers;
- no travel-history profile;
- no cloud synchronization;
- no cross-device sharing.

The backend receives public SL identifiers and filters needed for each request. A saved site and routine can still reveal a travel pattern, so logs must avoid retaining complete routine behaviour or associating requests with an identifiable person unnecessarily.

Security requirements:

- HTTPS only;
- strict query validation;
- safe, consistent errors;
- upstream timeouts;
- request-rate protection;
- no stack traces or configuration details in responses;
- environment-based configuration;
- dependencies kept current;
- platform-neutral contract tests.

Although the current upstream APIs are keyless, the backend must remain capable of protecting credentials if a future upstream service requires them.

---

19. Testing and release requirements

Backend tests

- request validation and error envelopes;
- site-search ranking, Swedish characters, and result limits;
- upstream-to-normalized serialization;
- departure identity;
- nullable expected time;
- cancellation from state and trip deviation;
- unknown state preservation;
- Swedish disruption-message selection and fallback;
- cache expiry and in-flight deduplication;
- timeout and upstream failure mapping;
- Stockholm daylight-saving-time transitions.

Android tests

- Room insert, update, delete, and observation;
- routine enable, disable, and pause behaviour;
- DTO-to-domain timestamp conversion;
- effective-time and countdown calculation;
- filtering by site, line, mode, and direction code;
- ViewModel behaviour where implemented;
- scheduler and notifier interfaces with fakes;
- stale and offline presentation.

Release checks

- Android build, lint, and unit tests;
- backend type-check, lint, tests, and production build;
- Vercel health and cache-header checks;
- real-device notification and lock-screen testing;
- scheduling tests across reboot and battery restrictions;
- attribution and terms review;
- no secrets or local environment files committed;
- no unrelated screenshots or generated build output in Git.

---

20. Success criteria

The MVP is successful when a user can configure a routine once and then, on subsequent selected weekdays:

- see the correct site, line, and direction automatically;
- see current upcoming departures in one Android notification;
- see the next departure replace the previous one;
- see relevant disruptions without unrelated messages;
- see cancellations and stale-data states clearly;
- have the notification disappear when the active period ends;
- use the feature without reopening the application each morning.

Technical success includes:

- reliable scheduled activation on supported Android devices;
- reasonable battery use;
- correct timestamp and daylight-saving-time behaviour;
- stable platform-neutral backend responses;
- compliance with upstream fair-use and attribution requirements;
- no unnecessary collection of personal information;
- no dependency on an exposed mobile API key.

---

21. Short project description

Blick is an Android-first scheduled departure display for regular SL commuters. Users choose a site, transport mode, line, direction, weekdays, and time window. During that period, upcoming departures and relevant disruptions appear automatically in one calm, updating Android notification. The platform-neutral backend is designed to support a planned native iPhone client and possible future household displays without expanding the initial MVP.